

## Homework 3

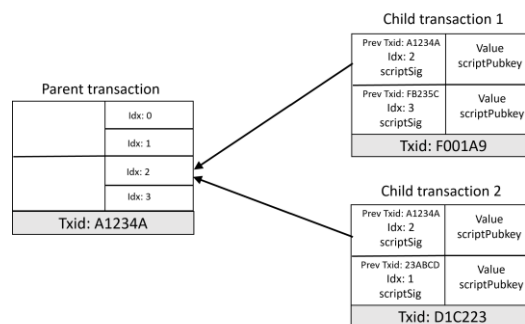
**Deadline: 11:55 pm April 12, 2026**

**General Instructions:** You may partner with up to three other students (**a group of max 4**) to submit a single write-up in a single group. We encourage discussion with other students, even if they are not in your group. Each student must build a good understanding of all the questions even if they discuss and collaborate with others. Merely dividing the problems among group members and putting those together before submission will severely impact your learning, and we **strongly advise against it**. Furthermore, while it is okay to use online resources, including **Generative AI** such as **ChatGPT**, for learning purposes, **blatant copying** or **mindlessly querying** the Generative AI is **forbidden**. If there is any confusion or questions, post those on Whatsapp group or see your TA or Instructor.

**Submission Instructions:** Typed or hand-written solutions (only in PDF format) can be submitted on LMS before the deadline. Email submissions WILL NOT be accepted. There will be NO late days or extensions. *Only one submission per group is needed; the first page must clearly mark the names and students IDs of all group members.*

### Question 1 [7 marks]

Consider the following image:



CS3812/EE3812 – Spring 2024

10

Are both of the transactions valid? Why and why not? If any of the two transactions is invalid, explain how the UTXO model inherently prevents this problem.

#### Solution:

- The parent transaction (Txid: A1234A) has four outputs: Idx: 0, Idx: 1, Idx: 2, and Idx: 3.
- Child transaction 1 (Txid: F001A9) uses:

- Input 1: Prev Txid: A1234A, Idx: 2
- Input 2: Prev Txid: FB235C, Idx: 3
- Child transaction 2 (Txid: D1C223) uses:
  - Input 1: Prev Txid: A1234A, Idx: 2
  - Input 2: Prev Txid: 23ABCD, Idx: 1

### Analysis:

- Both Child 1 and Child 2 attempt to spend the same UTXO: Prev Txid: A1234A, Idx: 2.
- This means they are both referencing the same unspent output from the parent transaction.
- As a result, if one transaction is included in a block and spends this UTXO, the other transaction will become invalid because the referenced output is no longer unspent.

Each transaction is **individually valid** if considered in isolation. However, they **cannot both be valid simultaneously** because they attempt to spend the same UTXO. The UTXO model inherently prevents this double-spend: once one transaction is confirmed and the output is spent, any other transaction trying to spend it will be rejected by the network. Double spending in the UTXO is prevented by the following ways:

When a new transaction is broadcast to the network, nodes (participants in the Bitcoin network) independently verify the transaction against the blockchain's history. A transaction is only considered valid if it spends UTXOs that are both unspent and valid themselves, ensuring that no bitcoin can be spent twice.

Bitcoin uses a proof-of-work (PoW) consensus mechanism where miners compete to add new blocks to the blockchain. Each block contains a set of transactions that the network has agreed upon. Once a transaction is included in a block and this block is added to the blockchain, it is considered confirmed. The deeper the block is in the blockchain (the more blocks are added on top of it), the more irreversible the transaction becomes. This layered consensus mechanism makes it extremely difficult to alter the transaction history to facilitate double-spending.

The blockchain acts as a public ledger, maintaining a transparent and chronological history of all transactions. This transparency ensures that the entire network can validate ownership history and prevent double spending by checking that each UTXO is only spent once.

### Question 2 [10 marks]

Explain how multi-signature (multisig) transactions are created in the Bitcoin network. How did Segwit improve the handling of multi-sig transactions? Discuss the differences in transaction size and verification process. Also explain how segwit improved on the security of multisig transactions.

**Solution:** Multisignature (multisig) transactions require multiple signatures for authorization before funds can be spent. Traditionally, these transactions included all the required signatures within the scriptSig part of the transaction. This approach had several drawbacks:

Each signature contributed to the overall transaction size, leading to higher fees for multisig transactions.

Since signatures are part of the transaction data used to calculate the transaction ID, altering the signatures could change the transaction ID without invalidating the transaction, posing security and operational risks.

With the introduction of SegWit, the handling of multisig transactions was significantly improved:

SegWit moved signatures (witness data) outside of the traditional transaction structure into a separate witness field. This solved the malleability issue (since the transaction ID is calculated without the witness data) and reduced the size of multisig transactions when calculated for fee purposes, resulting in lower fees.

By separating the signatures from the main transaction data, the verification process became more efficient. SegWit nodes can process and validate transactions faster because the witness data is handled separately.

SegWit's design improvements around multisig transactions enhanced the security by fixing transaction malleability and made the verification process more efficient, directly benefiting the scalability of the network and reducing transaction fees for users engaging in multisig transactions.

### **Question 3 Segwit Multisig Account [24 pts]:**

Eman, Namwar, and Zaeem have locked 3 BTC in 'eman-namwar-zaeem' joint account by creating a segwit 'pay to witness script hash' (p2wsh) transaction. Assume that the txid of the locking transaction is TXABC123 and it has two outputs. The funds are locked at the first output (index 0), while a 32 B hash of an important document called 'Project Sigma' is stored at the second output (index 1).

**a)** Let <sh> denote the script hash and <psigh> denote the hash of Project Sigma. Write down the complete scripts for the two outputs of this transaction. (4 pts)

**Solution:** Output script for index 0:

OP\_0 <sh>

Output script for index 1:

OP\_RETURN <psigh>

**b)** What will be the increase in UTXO database size once the txid TXABC123 is confirmed and added to the blockchain? (3 pts)

**Solution:** Size of output script at index 0: 1B (for OP\_0) + 1B (for script hash length) + 32B (for script hash) = 34B.

Size of UTXO entry at index 0: 8B (for value) + 1B (for script length) + 34B (for script) = 43B.

The second output uses OP\_RETURN, so it is not stored in the UTXO set.

**Increase in UTXO database size: 43B.**

**c)** What is the advantage of storing the hash of Project Sigma on the blockchain? (2 pts)

**Solution:** The data receives a blockchain timestamp when included in a block, proving that Eman, Namwar, and Zaeem knew about Project Sigma at or before the block's confirmation time, without revealing the document itself.

**d)** Suppose Eman and Namwar have to pay 0.8 BTC to Sana. The spending rules of txid TXABC123 require 2 of 3 signatures. Eman and Namwar create a new segwit spending txn with txid TXDEF456, which pays 0.8 BTC to Sana and locks the remaining funds in the 'eman-namwar-zaeem' joint account with the same spending rules as txid TXABC123. Write the complete scripts of the input including the witness field and the scripts of all the outputs of txid TXDEF456. Assume <sigE> and <sigN> represent the signatures of Eman and Namwar and <pkE>, <pkN>, and <pkZ> are the public keys of Eman, Namwar and Zaeem. The pubkey hash of Sana is <pkhS>. Assume the transaction fee is 0.1 BTC. (5 pts)

**Solution: Input:**

Previous txid = TXABC123, index = 0

scriptSig = empty

**Witness field:**

OP 0 <sigE> <sigN> <serialized script>

where:

< serialized script >= OP 2\_ <pkE> <pkN> <pkZ> OP 3 OP\_CHECKMULTISIG

**Outputs:**

- Output 0: value = 0.8 BTC, script: OP\_0 <pkhS>
- Output 1: value = 2.1 BTC, script: OP\_0 <sh>

Note: 2.1 BTC remains in the joint account since 0.1 BTC is paid as a transaction fee.

**e)** Suppose a segwit node wants to validate txid TXDEF456. Assuming that the transaction is valid, explain all the steps involved in the validation procedure for the segwit node. (3 pts)

**Solution:**

- a. The node inspects the parent output script and identifies it as a P2WSH script.
- b. It retrieves the witness field, including signatures and serialized redeem script.
- c. It hashes the serialized script and verifies that it matches <sh>.

- d. The node executes the script and verifies the two signatures are valid.
- e. It ensures that the sum of outputs is less than or equal to the input amount.

**f)** Suppose a non-segwit node (old node) wants to validate txid TXDEF456. Assuming that the transaction is valid, explain all the steps involved in the validation procedure for the old non-segwit node. (3 pts)

**Solution:**

- a. The old node ignores the witness field and sees an empty scriptSig.
- b. It directly executes the parent output script (OP\_0 <sh>).
- c. Since a non-zero value remains on top of the stack, this check passes.
- d. The node confirms that the total output value is less than or equal to the input value.

**g)** Suppose Sana knows about the transaction malleability vulnerability of Bitcoin transactions. She also knows a corrupt miner who can malleate transactions. Will Sana and her miner friend succeed in malleating txid TXDEF456? Explain your answer. (2 pts)

**Solution:** No. SegWit transactions exclude witness data from the transaction ID calculation. Since signatures are in the witness field, changing them does not alter the txid. Therefore, Sana and the miner cannot malleate txid TXDEF456.

**h)** Suppose Zain knows the spending rules of 'eman-namwar-zaeem' joint account and also knows all the public keys. With this information, Zain can create a serialized script whose hash equals the script hash <sh> at output 0 of txid TXABC123. Assuming that Zain is malicious, can he create a transaction to unlock the funds in the joint account? (2 pts)

**Solution:** No. Even if Zain reconstructs the serialized script and matches the script hash, he cannot produce valid signatures without the private keys of Eman, Namwar, or Zaeem. Thus, he cannot spend the funds. The hidden script provides additional privacy but not additional security beyond the signature requirement.

**Question 4 Multiple Transactions [12 pts]** Suppose the following transactions are broadcasted **simultaneously** on the blockchain network and all become available to a miner.

**Transaction hash: ABC3**

| Inputs                        | Outputs            |
|-------------------------------|--------------------|
| Some valid input worth 10 BTC | Alice → 3.95 BTC   |
|                               | Bob → 1.95 BTC     |
|                               | Charlie → 3.90 BTC |

**Transaction hash: BD9E**

| Inputs                                       | Outputs          |
|----------------------------------------------|------------------|
| Prev hash: ABC3<br>Idx: 0<br>Signed by Alice | Dianne → 3.9 BTC |

**Transaction hash: C4B7**

| Inputs                                         | Outputs                             |
|------------------------------------------------|-------------------------------------|
| Prev hash: ABC3<br>Idx: 1<br>Signed by Bob     | Fring → 1.90 BTC                    |
| Prev hash: ABC3<br>Idx: 2<br>Signed by Charlie | Bob → 1.0 BTC<br>Charlie → 2.85 BTC |

**Transaction hash: D743**

| Inputs                                         | Outputs          |
|------------------------------------------------|------------------|
| Prev hash: ABC3<br>Idx: 0<br>Signed by Alice   | George → 4.0 BTC |
| Prev hash: C4B7<br>Idx: 2<br>Signed by Charlie | Hank → 2.60 BTC  |

The miner has the flexibility to pick the transactions in any order. What transaction order would maximize their mining fee? What would be that fee?

**a)** Identify all the invalid transactions (you should mention the transaction hashes, which are given). Also state a reason, why do you think a transaction is invalid.

**Solution:**

Both transaction BD9E and D743 attempt to spend the output locked to Alice in transaction ABC3. Hence, both transactions are invalid until one of them gets confirmed, making only the other one invalid.

**b)** Suppose the valid transactions make it to the mempool and a miner picks them up for inclusion in a block. How much fee would he receive from these transactions. Show your working.

**Solution:** Transaction ABC3 has an input of 10BTC and an output of  $3.95\text{BTC} + 1.95\text{BTC} + 3.90\text{BTC} = 9.8\text{BTC}$  so the fee is 0.20BTC.

Transaction BD9E has an input of 3.95BTC and an output of 3.9BTC so the fee is 0.05BTC.

Transaction C4B7 has an input of  $1.95 + 3.9\text{BTC} = 5.85$  and an output of  $1.90 + 1.0 + 2.85\text{BTC} = 5.75$ . The fee is 0.1BTC.

Transaction D743 has an input of  $3.95 + 2.85\text{BTC} = 6.8$  and an output of  $4 + 2.6\text{BTC} = 6.6$  so a fee of 0.20BTC.

Both transaction BD9E and D743 attempt to spend the output locked to Alice in transaction ABC3. As transaction D743 has a greater transaction fee, it is picked over transaction BD9E, giving us a total fee of  $0.2 + 0.2 + 0.1 = 0.5$  BTC.

### Question 5 Advanced SegWit Concepts [15 pts]

Segregated Witness (SegWit) was a major upgrade to Bitcoin introduced via a soft fork in 2017. It changed how transaction data is organized and validated, enabling scalability improvements and new features.

**a)** Explain how SegWit changes the structure of a Bitcoin transaction compared to a legacy transaction. Specifically, describe what is stored in the *witness* field and how it relates to the scriptSig and scriptPubKey (5 pts).

**Solution:** In a legacy Bitcoin transaction:

- Each input contains a scriptSig, which provides the unlocking script (e.g., a digital signature and public key) required to spend the previous output.
- The scriptPubKey in the previous output specifies the locking condition that the scriptSig must satisfy.

With **SegWit**, the digital signatures and related data are moved out of the main transaction structure into a separate field called the **witness**:

- The scriptSig in a SegWit input is typically empty or minimal (e.g., containing a redeem script hash).
- The unlocking data (signatures, public keys) are instead placed in the **witness field**.
- This witness data is not included when computing the transaction ID (txid), but is still required for validation.

This separation reduces the size of the base transaction, enables new script types (like P2WPKH and P2WSH), and improves efficiency.

**b)** Before SegWit, Bitcoin transactions were vulnerable to a form of *transaction malleability*. Explain what transaction malleability is, how it could be exploited, and how SegWit solves this problem. (5 pts)

**Solution: Transaction malleability** is the ability to alter the transaction ID (txid) without changing its validity.

Before SegWit:



- The txid was computed from the entire transaction, including the scriptSig.
- Because the scriptSig contains signatures (which are malleable by nature), an attacker could slightly modify the signature format, producing a **different** txid even though the transaction still spent the same coins and remained valid.

**Impact:** This was a problem for multi-step protocols (like payment channels or the Lightning Network), which relied on predictable transaction IDs.

**SegWit's solution:**

- The txid no longer includes the witness data (signatures) when being computed.
- Since signatures are the main source of malleability, excluding them makes the transaction ID stable once created.
- The witness is still verified during validation, but it doesn't affect the transaction's identity.

**c)** SegWit was deployed as a **soft fork** rather than a hard fork. Explain how this was achieved in terms of transaction compatibility. In your answer, describe how older nodes (that have not upgraded) interpret SegWit transactions and why consensus is still maintained. (5 pts)

**Solution:** SegWit was designed as a **soft fork** — meaning it added new rules without invalidating old ones. This was achieved using a clever encoding trick:

- SegWit transactions use new output types, such as **Pay-to-Witness-Public-Key-Hash (P2WPKH)** or **Pay-to-Witness-Script-Hash (P2WSH)**.
- These are embedded inside a **Pay-to-Script-Hash (P2SH)** output, which older nodes already support.
- Legacy nodes see the SegWit output as a regular P2SH output and accept it if the spending script hash matches, even though they don't understand the witness field.
- Upgraded nodes, however, parse the witness and enforce the new SegWit rules.

**Result:** Older nodes continue to see the blockchain as valid because all transactions follow the original consensus rules they understand. Meanwhile, upgraded nodes enforce the stricter SegWit rules, improving security and functionality without causing a chain split.

**Question 6 Merkle Tree and Hashing [11 pts]**

A Bitcoin block contains 2048 transactions. A lightweight SPV (Simplified Payment Verification) client wants to verify that a specific transaction is included in this block.

**a)** Calculate the exact number of hashes the SPV client needs to receive as proof to verify the transaction's inclusion in the block. Show your calculation. [3 pts]

**Solution:** The depth of a Merkle tree with 2048 transactions is calculated as  $\log_2(2048) = \log_2(2^{11}) = 11$  levels. To verify a single transaction, the SPV client needs one sibling hash at each level from the leaf to the root. Therefore, the number of hashes needed is **11 hashes**.

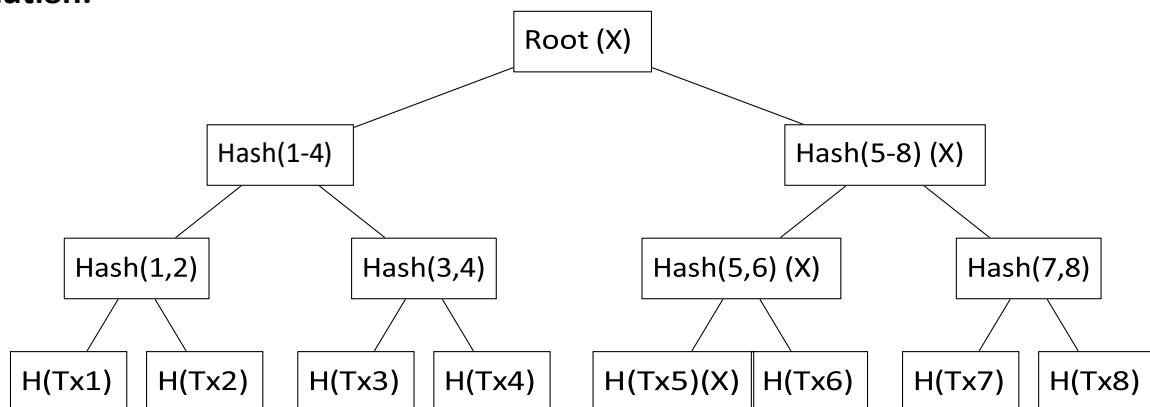
**b)** If the block size increases to contain 16,384 transactions, how many additional hashes would be needed for the Merkle proof compared to part (a)? Explain why Merkle trees provide logarithmic scalability. [4 pts]

**Solution:** For a block containing 16,384 transactions, the depth is  $\log_2(16,384) = \log_2(2^{14}) = 14$  levels, requiring 14 hashes for the proof. The additional hashes needed compared to part (a) is  $14 - 11 = 3$  hashes.

Merkle trees provide logarithmic scalability because when the number of transactions increases from 2,048 to 16,384 (an 8-fold increase), the proof size only grows from 11 to 14 hashes (an increase of just 3). This demonstrates that proof size grows as  $O(\log_2 n)$  rather than linearly with the number of transactions, making verification extremely efficient even for blocks containing thousands of transactions.

**c)** Suppose an attacker tries to create a fake Merkle proof by modifying transaction Tx5 in a block containing 8 transactions. Draw the Merkle tree and mark with an 'X' all the nodes that would have different hash values due to this modification. Explain why this makes fraud detection efficient. [4 pts]

**Solution:**



The nodes marked with (X) that have changed hash values are:  $H(Tx_5)$  (the leaf node containing the direct hash of the modified transaction), Hash(5, 6) (the parent of Tx5 and Tx6), Hash(5 – 8) (the grandparent), and the Merkle Root.

Fraud detection is efficient because any modification to Tx5 changes its hash, and this change propagates up the tree, altering all ancestor nodes. The final Merkle Root will differ from what is stored in the block header. Verification requires only  $O(\log_2 n)$  hashes to detect fraud, and the attacker cannot forge a valid proof without breaking SHA-256's collision resistance property, which is computationally infeasible.

### Question 7 UTXO Model [12 pts]:

Consider the following scenario involving Alice, Bob, and Charlie:

- UTXO<sub>1</sub>: Alice owns 5 BTC at (Txid: XYZ789, Index: 0)
- UTXO<sub>2</sub>: Alice owns 3 BTC at (Txid: ABC456, Index: 1)
- UTXO<sub>3</sub>: Bob owns 2 BTC at (Txid: DEF123, Index: 0)

**a)** Alice wants to pay Bob 6 BTC and Charlie 1.5 BTC in a single transaction. Design a complete valid transaction showing all inputs, outputs, and transaction fee. Explain why Alice needs to use both UTXOs and what happens to remaining funds. [3 pts]

**Solution:** Alice needs to use both UTXOs because the total payment required is  $6 + 1.5 = 7.5$

BTC, and neither UTXO alone is sufficient. In Bitcoin's UTXO model, outputs must be spent completely, you cannot spend partial amounts from a UTXO. The remaining funds after payments and (€) total  $8 - 7.5 - \epsilon = 0.50 - \epsilon$  BTC. This creates a "change output" (Output 2) that sends the remaining funds back to Alice, similar to receiving change when paying with cash. This change output becomes a new UTXO that Alice can spend in future transactions.

**b)** After the transaction in part (a) is confirmed, list all UTXOs added to and removed from the UTXO database. [2 pts]

**Solution:** Two UTXOs are removed from the database: (XYZ789, 0) containing 5 BTC belonging to Alice, and (ABC456, 1) containing 3 BTC belonging to Alice. Three UTXOs are added to the database: (NEW001, 0) containing 6 BTC for Bob, (NEW001, 1) containing 1.5 BTC for Charlie, and (NEW001, 2) containing change BTC for Alice as change.

**c)** Bob now wants to create two simultaneous transactions: T1 sending 3 BTC to Charlie, and T2 sending 3 BTC to Alice. Both transactions reference Bob's UTXO from Alice's payment in part (a), which contains 6 BTC at (NEW001, 0). Are both transactions valid or invalid (explain), what happens when they are broadcast to the network, and which transaction gets confirmed (what are the determining factors behind a transaction getting confirmed). [5 pts]

**Solution:** In isolation, both transactions are technically valid: each has valid signatures from Bob, correct input references, and output sums that do not exceed input sums. However, together they represent a double-spend attempt because both try to spend the same UTXO (NEW001, 0). Only one can be confirmed on the blockchain.

When both transactions are broadcast to the network, different nodes may receive them in different orders. A node that receives T1 first will validate it, add it to its mempool, and then reject T2 when it arrives because the input UTXO is already marked as spent in the mempool. Conversely, a node receiving T2 first will accept T2 and reject T1. This creates a temporary network inconsistency where some nodes have T1 in their mempool and others have T2.

The transaction that gets included in a block first wins the race. The determining factors include: (1) miner selection based on transaction fees (miners typically prefer transactions with higher fee rates), (2) network propagation (the transaction that reaches more miners first has a higher chance), and (3) mining luck (whichever miner finds the next block determines the winner). Once one transaction is confirmed in a block, all nodes remove the losing transaction from their mempools as it becomes permanently invalid. The UTXO model prevents double-spending by ensuring each UTXO can only be spent once.

**d)** Explain why the UTXO model prevents double-spending better than an account-based model. [2pts]

**Solution:** The UTXO model prevents double-spending more effectively than an account-based model for several reasons. First, the UTXO model enforces atomic consumption: each output is spent entirely in one transaction, and once spent, it is removed from the UTXO set, creating a binary state where a UTXO either exists or doesn't exist. In contrast, an account-based model requires checking current balance state, which is a more complex continuous value that could be anywhere from zero to the maximum balance. Second, the UTXO model

enables parallelizable validation: transactions spending different UTXOs can be validated independently and concurrently, whereas in an account-based model, transactions affecting the same account must be processed sequentially to ensure the balance never goes negative. Third, the UTXO model creates clear dependency chains, making fraud detection easier through explicit lineages of transactions.

### **Question 8 Merkle Trees and UTXO [13 pts]:**

Bitcoin's design uses Merkle trees in block headers to commit to transactions, while nodes maintain a separate UTXO database.

**a)** Explain why Bitcoin uses two separate data structures (Merkle tree of transactions in blocks and UTXO database) rather than just storing all unspent outputs in a Merkle tree in the block header. Discuss at least two practical reasons related to efficiency and functionality. [5 pts]

**Solution:** Bitcoin uses two separate data structures for several practical reasons related to efficiency and functionality. First, regarding efficiency and fast transaction validation, the UTXO database provides  $O(1)$  lookup time with a hash table (or  $O(\log n)$  with a tree structure), allowing nodes to quickly check if an input UTXO exists and is unspent. If UTXOs were stored only in a Merkle tree, validation would require  $O(n)$  scanning or maintaining an additional index, and every new block would necessitate rebuilding the entire Merkle tree at  $O(n \log n)$  cost. Since Bitcoin validates approximately 2,000 transactions per block every 10 minutes, and nodes continuously validate mempool transactions, fast UTXO lookup is critical for network performance. With the current UTXO set containing over a large amount of entries, efficient access is essential.

Second, the two structures serve fundamentally different purposes. The Merkle tree provides historical proof, answering the question "Was transaction T included in block B?" It creates an immutable record of all transactions, enables lightweight SPV client verification, and provides cryptographic commitment to block contents. In contrast, the UTXO database tracks current state, answering "Can this transaction be spent right now?" It tracks only currently spendable outputs, efficiently answers existence queries, supports fast updates (additions and deletions), and prevents double-spending in real-time. For example, an SPV client uses Merkle proofs to verify a payment was received without downloading the full block, while a node uses the UTXO database to validate an incoming transaction in under one millisecond.

Third, regarding storage efficiency, the full blockchain with Merkle trees contains all transactions ever made and has grown to approximately 600 GB, including both spent and unspent outputs. The UTXO database contains only unspent outputs and occupies approximately 8 GB. If we only stored UTXOs in a Merkle tree without the full blockchain, we would lose historical transaction data, be unable to verify old SPV proofs, lose the ability to audit blockchain history, and be unable to reconstruct the UTXO set from the genesis block. Fourth, the update patterns differ significantly: Merkle trees are append-only and immutable (each block has its own Merkle tree that is never modified after creation), while the UTXO database requires frequent updates (every transaction removes spent UTXOs and adds new ones, with approximately 4,000 deletions and 5,000 insertions per block every 10 minutes), necessitating an efficient mutable data structure.

**b)** An SPV client receives a Merkle proof showing that transaction  $Tx_K$  is in Block 800,000. However, this SPV client later wants to verify that  $Tx_K$  hasn't been spent yet. Explain why the Merkle proof alone is insufficient for this verification, and what additional information the client would need. What security assumption must the SPV client make? [8 pts]

**Solution:** The Merkle proof alone is insufficient because it only demonstrates that  $Tx_K$  exists in block 800,000 at the time the block was created, but it does not provide any information about whether the outputs of  $Tx_K$  have been spent in subsequent blocks. For example, if  $Tx_K$  created an output at  $(Tx_K, 0)$  with 10 BTC to Alice in block 800,000, and then a later transaction  $Tx_M$  in block 800,500 spent that output, the Merkle proof for  $Tx_K$  in block 800,000 would still be valid, it permanently proves that  $Tx_K$  was included in that historical block. However, this proof provides no information about the current spendability status of the outputs. The Merkle proof is static and historical; it does not expire when outputs are spent, and the blockchain's append-only nature means block 800,000 will always contain  $Tx_K$  regardless of future spending events.

To verify that  $Tx_K$  outputs haven't been spent, the SPV client needs additional information. The most practical approach is to query a full node to check whether  $(Tx_K, output\_index)$  exists in the current UTXO set. Alternatively, but much more expensively, the client could scan all subsequent blocks from 800,001 onwards to check if any transaction spent the outputs of  $Tx_K$ . Some SPV clients rely on trusted blockchain explorer services to provide this information.

The SPV client must make important security assumptions: it must trust that the full nodes it queries are honest and not lying about the UTXO state, assume that the majority of hash power is honest (ensuring the longest chain is valid), and verify that the block containing  $Tx_K$  has sufficient confirmations to be considered irreversible. This is a fundamental limitation of SPV clients: they sacrifice the ability to independently verify unspent status (which requires maintaining the full UTXO database) in exchange for lightweight operation that requires only block headers and Merkle proofs.